

웹에서 대화해보기



Contents

1 반려로봇 완성하기

1.1 목표

1.2 전체 코드 구조

1.3 flask를 사용하기 위해 로봇에 설치

2 robot_emotion.py

2.1 먼저 설정과 초기화

2.2 led 색 정의

2.3 play rainbow

2.4 led emotion

2.5 lcd emotion

2.6 motor emotion

2.7 emotion 통합

3 openai client

3.1 환경 설정 및 초기화

3.2 시스템 프롬프트

3.3 gpt 대화 req 부분

3.4 gpt 대화 response 처리 부분

4 app.py

4.1 초기화

4.2 flask 초기화, 대화 기록용 변수 history 선언

4.3 웹사이트 첫 페이지를 보여주는 부분

4.4 브라우저에서 메시지를 보내는 api 엔드포인트

4.5 GPT API 호출

4.6 감정에 맞게 로봇 움직임

5 index.html

5.1 핵심부분인 Java Script 부분 중 사용자 메시지를 전송할때 호출될 sendMsg 함수

5.2 사용자가 입력한 문장을 채팅창에 표현

5.3 flask 서버의 /chat API로 전송하는 부분

5.4 서버의 응답 - text와 emotion을 받아서 출력

6 실행

6.1 app.py를 실행하면

6.2 이제 웹에서 대화할 수 있다

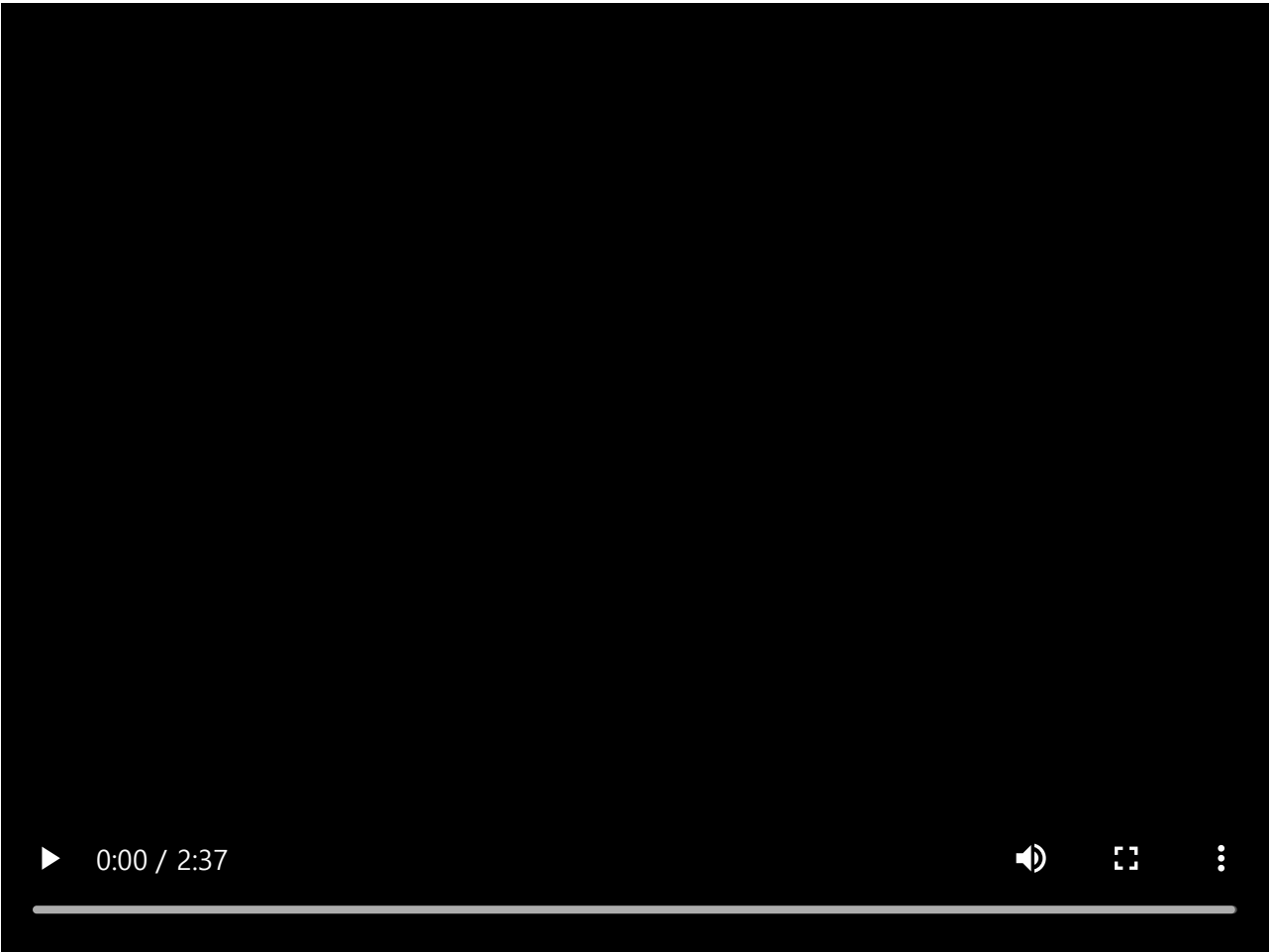
6.3 끝나면 정리~

웹에서 대화해보기

Exported from Confluence · <https://pinkwink.atlassian.net>

반려로봇 완성하기

목표



전체 코드 구조


```

robot_app/
|
├─ app.py           ← Flask 웹 서버 (대화/감정 처리)
├─ robot_emotion.py ← LED/LCD/Motor 제어 및 감정 표현
├─ openai_client.py ← OpenAI API + 감정 분석
|
├─ static/
|   └─ style.css    ← 핑크톤 웹 CSS
|
└─ templates/
    └─ index.html   ← 매우 심플한 대화 UI

```

flask를 사용하기 위해 로봇에 설치

- pip3 install flask

robot_emotion.py

먼저 설정과 초기화

```

1  # robot_emotion.py
2
3  from pinky_lcd.pinky_lcd import LCD
4  from pinkylib import LED, Motor
5  from PIL import Image, ImageSequence
6  import time
7
8  led = LED()
9  lcd = LCD()
10 motor = Motor()
11 motor.enable_motor()

```

led 색 정의

```

13 # LED 색 정의
14 RED=(255,0,0); YELLOW=(255,255,0); BLUE=(0,0,255)
15 GREEN=(0,255,0); INDIGO=(75,0,130); WHITE=(255,255,255)
16 RAINBOW=[RED,YELLOW,GREEN,BLUE,INDIGO]

```

play rainbow

```

18 def play_rainbow(dt=0.08):
19     for c in RAINBOW:
20         led.fill(c); time.sleep(dt)

```

led emotion

```
22 def led_emotion(e):
23     if e=="angry": led.fill(RED)
24     elif e=="happy": led.fill(YELLOW)
25     elif e=="fun": play_rainbow()
26     elif e=="sad": led.fill(BLUE)
27     elif e=="interest": led.fill(GREEN)
28     else: led.fill(WHITE)
```

lcd emotion

```
30 def lcd_emotion(e):
31     emotion_folder_path = '/home/pinky/pinky_pro/src/pinky_pro/pinky_emotion/emotion/'
32     emotion_path = f"{emotion_folder_path}/{e}.gif"
33     gif = Image.open(emotion_path)
34
35     for f in ImageSequence.Iterator(gif):
36         lcd.img_show(f)
```

motor emotion

```
38 def motor_emotion(e):
39     if e in ["happy", "fun"]:
40         for _ in range(3):
41             motor.move(50, -50); time.sleep(0.3)
42             motor.move(-50, 50); time.sleep(0.3)
43     elif e=="sad":
44         for _ in range(2):
45             motor.move(40, 40); time.sleep(0.3)
46             motor.move(-40, -40); time.sleep(0.3)
47     motor.stop()
```

emotion 통합

```
49 def run_emotion(e):
50     led_emotion(e)
51     lcd_emotion(e)
52     motor_emotion(e)
```

openai client

환경 설정 및 초기화

```

2  import os, json
3  from dotenv import load_dotenv
4  from openai import OpenAI
5
6  load_dotenv()
7
8  client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
9  EMOTIONS = ['angry', 'basic', 'bored', 'fun', 'happy', 'hello', 'interest', 'sad']

```

시스템 프롬프트

```

11  SYSTEM_PROMPT = f"""
12  Always reply ONLY JSON:
13  {{
14      "assistant_message": "<answer in Korean>",
15      "emotion": "<one of: {'', '.join(EMOTIONS)}>"
16  }}
17  Infer emotion from user's message. If unclear use "basic".
18  """

```

gpt 대화 req 부분

```

20  def ask_gpt(text, history):
21      msgs = [{"role": "system", "content": SYSTEM_PROMPT}] + history
22      msgs.append({"role": "user", "content": text})
23
24      res = client.chat.completions.create(
25          model="gpt-4.1-mini",
26          messages=msgs,
27          temperature=0.4
28      )

```

gpt 대화 response 처리 부분

```

20  def ask_gpt(text, history):
30      raw = res.choices[0].message.content
31      try:
32          data = json.loads(raw)
33          msg = data["assistant_message"]
34          emo = data["emotion"]
35      except:
36          msg, emo = raw, "basic"
37
38      if emo not in EMOTIONS: emo="basic"
39
40      history.append({"role": "user", "content": text})
41      history.append({"role": "assistant", "content": raw})
42
43      return msg, emo, history

```

app.py

초기화

```
2 from flask import Flask, render_template, request, jsonify
3 from openai_client import ask_gpt
4 from robot_emotion import run_emotion
```

flask 초기화, 대화 기록용 변수 history 선언

```
6 app = Flask(__name__)
7
8 history = []
```

웹사이트 첫 페이지를 보여주는 부분

```
10 @app.route("/")
11 def index():
12     return render_template("index.html")
```

- 사용자가 웹브라우저에서 라즈베리파이에 접속하면 `/` 경로로 연결된다.
- `templates/index.html` 파일을 브라우저에 띄워준다.
- 우리가 만든 핑크톤 채팅 UI 화면이 바로 여기서 렌더링된다.

브라우저에서 메시지를 보내는 api 엔드포인트

```
14 @app.route("/chat", methods=["POST"])
15 def chat():
16     user_text = request.json["text"]
```

- 브라우저가 메시지를 보낼 때 `/chat` 에 POST로 전송한다.
- `request.json["text"]`
→ 사용자가 입력한 실제 메시지를 읽어온다.

GPT API 호출

```
14 @app.route("/chat", methods=["POST"])
15 def chat():
16     user_text = request.json["text"]
17     reply, emotion, updated = ask_gpt(user_text, history)
18     run_emotion(emotion)
19     return jsonify({"reply": reply, "emotion": emotion})
```

- `reply` — GPT가 대답한 메시지

- **emotion** — GPT가 판단한 감정
- **updated** — 업데이트된 history(대화 기록)

감정에 맞게 로봇 움직임

```
14 @app.route("/chat", methods=["POST"])
15 def chat():
16     user_text = request.json["text"]
17     reply, emotion, updated = ask_gpt(user_text, history)
18     run_emotion(emotion)
19     return jsonify({"reply": reply, "emotion": emotion})
```

- 그리고 웹브라우저에게 응답과 감정을 보내줌

index.html

핵심부분인 Java Script 부분 중 사용자 메시지를 전송할때 호출될 sendMsg 함수

```
19 <script>
20 async function sendMsg() {
21     const text = document.getElementById("user-input").value;
```

사용자가 입력한 문장을 채팅창에 표현

```
19 <script>
20 async function sendMsg() {
23     document.getElementById("chat-box").innerHTML +=
24     `<div class='user'>👤 ${text}</div>`;
```

flask 서버의 /chat API로 전송하는 부분

```
20 async function sendMsg() {
26     const res = await fetch("/chat", {
27         method: "POST",
28         headers: {"Content-Type": "application/json"},
29         body: JSON.stringify({text})
30     });
```

서버의 응답 - text와 emotion을 받아서 출력

```
20 async function sendMsg() {
32     const data = await res.json();
33
34     document.getElementById("chat-box").innerHTML +=
35     `<div class='bot'>🤖 ${data.reply} <span class='emo'>(${data.emotion})</span></div>`;
```

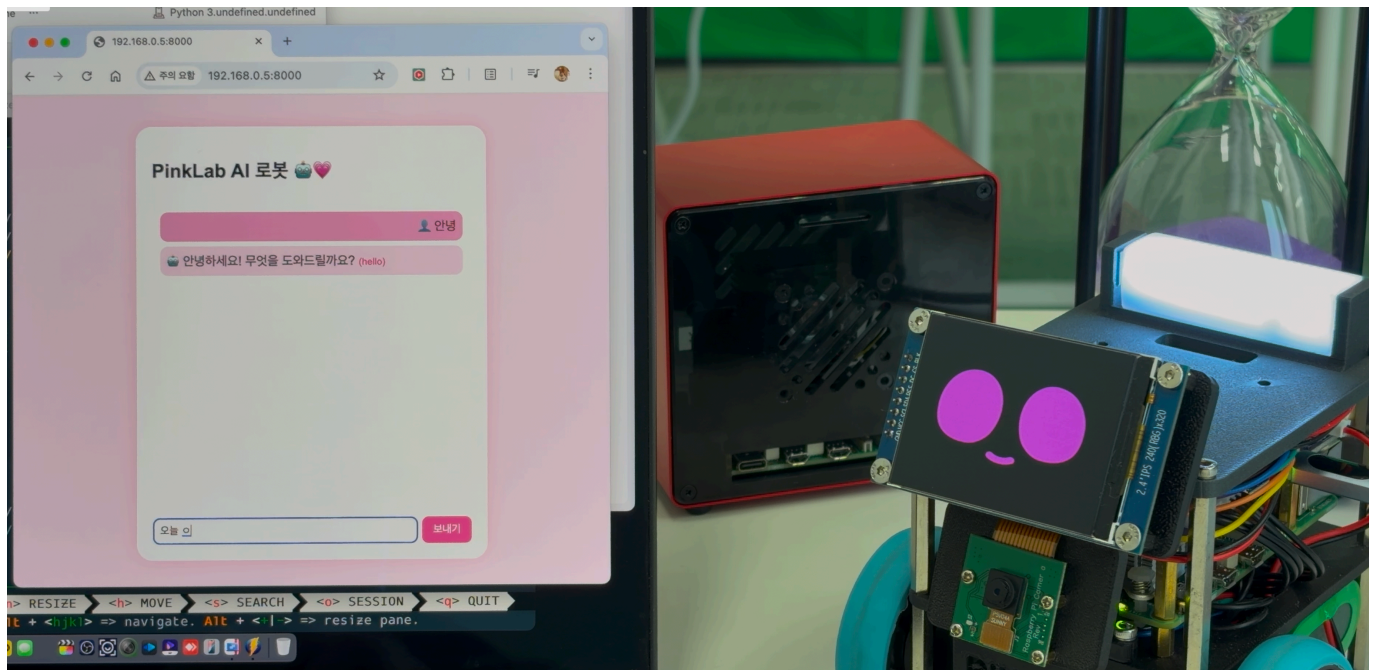
실행

app.py를 실행하면

Code

```
1 python3 app.py
```

이제 웹에서 대화할 수 있다



끝나면 정리~

app.py를 실행한 터미널을 종료하고, 정리되지 않은 자원은 따로 정리해주자.

```

In [ ]: 1 from PIL import Image
2
3
4 def safe_call(name, func):
5     try:
6         func()
7         print(f"[OK] {name}")
8     except Exception as err:
9         print(f"[WARN] {name} 실패: {err}")
10
11
12 def main():
13     def led_off():
14         from pinkylib import LED
15         led = LED()
16         if hasattr(led, "clear"):
17             led.clear()
18         else:
19             led.fill((0, 0, 0))
20
21     def lcd_off():
22         from pinky_lcd.pinky_lcd import LCD
23         lcd = LCD()
24         if hasattr(lcd, "clear"):
25             lcd.clear()
26         else:
27             black = Image.new("RGB", (320, 240), (0, 0, 0))
28             lcd.img_show(black)
29
30     def motor_stop():
31         from pinkylib import Motor
32         motor = Motor()
33         if hasattr(motor, "enable_motor"):
34             motor.enable_motor()
35         motor.stop()
36
37     safe_call("motor.stop()", motor_stop)
38     safe_call("led off", led_off)
39     safe_call("lcd off", lcd_off)
40
41     print("완료: LED/LCD/Motor 종료 처리를 시도했습니다.")
42
43
44 if __name__ == "__main__":
45     main()

```



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio